



Delay Control and Parallel Admission Algorithms for Real-Time Anycast Flow*

WEIJIA JIA

itjia@cityu.edu.hk

BO HAN

Department of Computer Engineering and Information Technology, City University of Hong Kong, Kowloon, Hong Kong, SAR China

CHUANLIN ZHANG

Department of Mathematics, Jinan University, Guangzhou 510632, People's Republic of China

WANLEI ZHOU

School of Information Technology, Deakin University, 221 Burwood Highway, Burwood, VIC 3125, Australia

Abstract. An anycast flow is a flow that can be connected to any one of the members in a group of designated (replicated) servers (called anycast group). In this paper, we derive a set of formulas for calculating the end-to-end delay bound for the anycast flows and present novel admission control algorithms for anycast flows with real-time constraints. Given such an anycast group, our algorithms can effectively select the paths for anycast flows' admission and connection based on the least end-to-end delay bounds evaluated. We also present a parallel admission control algorithm that can effectively calculate the available paths with a short delay bound for different destinations in the anycast group so that a best path with the shortest delay bound can be chosen.

Keywords: anycast flow, delay control, parallel admission algorithm

1. Introduction

Unlike datagram communication, a flow-oriented communication has to go through an admission process in which an application makes a request with certain real-time requirements, to the network for establishing a flow between a source and an anycast group. An anycast flow is a sequence of packet flow that can be connected to any one of the members in a group of designated receivers (called anycast group). An anycast flow can be admitted only if sufficient network resources are available so that the required real-time deadline can be adhered. The admission control plays a critical role in meeting with the real-time requirement of anycast flows. By real-time we mean that every packet of the anycast flow is delivered from its source to the same destination in a group of designated recipients within a predefined end-to-end deadline. Packets delivered beyond this end-to-end deadline are considered useless.

*The work is supported by Hong Kong RGC China grant nos.: CityU 1039/02E, CityU 1055/01E and CityU strategic grant nos.: 7001587 and 7001446.

There are many applications that require communication services in the form of anycast flows, e.g., e-transaction, e-banking, down-loading, and up-loading etc. Using anycast communication services may considerably simplify these applications. Multiple mirrored (replicated) servers of the service providers can share a single anycast address. Applications may simply send their information flows with the anycast address in order to upload or download information to or from these multiple replicated servers. The anycast service has been defined as a standard service in the latest version of IPv6 [10]. It was determined that anycast addresses are allocated from the unicast address space with any of the defined unicast address format [16]. The implication of an anycast service supported at the application layer was explored in [3]. A framework for scalable global IP anycast was proposed in [21]. An anycast protocol and its integration with other routing approaches were developed in [9, 19, 20, 22, 24]. Admission control for unicast and multicast packets has been investigated widely in many well-know systems as described in [4, 7, 8, 12]. Distributed admission control for anycast flows with quality of service (QoS) requirements was developed recently in [23].

To provide a better QoS for anycast flow transmission, especially, the end-to-end delay, in this paper, we propose a novel algorithm in calculation of end-to-end delay bound for anycast flows using network calculus [7, 8]. However, our approach differs from the network calculus in the sense that an anycast flow may be connected to any one of replicated servers in an anycast group. Thus, a “best” route may be selected for the connection of an anycast flow if the delay constraint of the flow can be satisfied. We propose a novel admission control algorithm based on the delay bound evaluation and extend the algorithm into a parallel admission control algorithm thus the path with the shortest delay bound can be selected effectively.

2. Related work

Because more and more applications demand anycast services, in the latest version of IPv6, anycast has been defined as a standard service [11]. The problems pertaining to anycast can be divided into two classes: management methods at the application layer for using anycast services, and procedures and protocols at network layer for routing and addressing anycast messages. In [17], it was determined that anycast addresses are allocated from the unicast address space. An additional set of reserved anycast addresses within each subnet prefix has been also defined in [11]. Some subnet routers can be assigned the reserved anycast addresses that represent all routers within a subnet prefix. A framework for scalable global IP anycast was proposed in [21]. Early work using anycast for locating the nearby replicated server has been proposed in [14]. Anycasting paradigm at the application layer has been examined in [3, 25] in which Zegura et al. have provided an efficient service that uses an anycast resolver to map an anycast domain name and a selection criteria into an IP address. The implication of an anycasting replicated service was also explored through server push and probe approach. Anycast has been partially defined as private network-network interface (PNNI) standardization. Frequently used anycast addresses identify access points to key services that are generally distributed on different switches due to high availability; multiple instances of the same address can

also exist in the network. Routing to these addresses is similar to routing to exterior addresses. IBM implementation includes the cost of the last hop in computing paths, defined as the cost of the non-PNNI link on which the internal address is defined. Taking this last link into account allows the IBM implementation to compute the best possible paths to these addresses. However, PNNI does not specify how to select a server or path [2]. Q-OSPF [13] is based on routing tables that reflect the global network state. The focus of Q-OSPF is on the algorithms used to compute QoS routes and on the necessary modifications to OSPF to support this function, e.g., the information needed, its format, how it is distributed, and how it is used by the QoS path selection process [1]. However, hop-to-hop OSPF support is indispensable. In [15], QoS routing for anycast communication is discussed in the context of DiffServ networks in which server selection and resource reservation are attained in an aggregated fashion rather than being driven by individual client demand.

3. Network and anycast traffic models

A network is modeled as a graph $H = N(S, E)$ of connected link servers, where S is a finite set of vertices in H , representing nodes (routers or switches) in the network concerned. E is a finite set of edges, representing the links between nodes. For the purpose of delay computation, we follow the standard practice and model a router like a server, where packets can experience queuing delays. Packets are typically queued at the output buffers, where they compete for the output link. We therefore model a router as a set of output link servers. All other servers (input buffers, non-blocking switch fabric, wires etc.) can be eliminated by appropriately subtracting the constant delays incurred on them from the deadline requirements of traffic. The link servers in set S are connected through either links in the network or paths within the routers, which both make up the set of edges E in the graph. We assume that all routers have N input links and all the output links with link capacity C bits per second. We use L to denote the maximal number of input edges to each server.

A stream of packets between one sender and one receiver is called a flow. An anycast flow (AF) is a sequence of packets that, in total, can be sent to any one of the members in a group of designated recipients [23]. We assume that all packets in AFs have the same length or the same upper bounds. Furthermore, we assume that anycast flows have a request on sequencing. That is, once the first packet in an AF is delivered to a member in the recipient group, all the consequent packets of the same flow should also be delivered to the same member. Anycast packets of a flow are transmitted along some (shortest) path from its source to one member in the group of designated recipients. We model a path as a list of link servers. For the incoming anycast packet, the router determines the output link by looking up its routing table and transports anycast packets to the proper output link, which, in turn, connects to the next hop. Anycast packets may exceed the capacity at the output link; thus, an anycast packet may suffer from some queuing delay at the output link. This delay depends on the load of the output link and the scheduling policy adopted at the output link. In the time interval with a length I , all flows (anycast or non-anycast) going through server k from input link j form a group of flows $G_{j,k}(I)$.

We define a group of anycast members as a set of servers that provide same kind of services. Therefore, a user may request any of the members for the connection of services by issuing anycast connection for anycast flow. It can be seen that anycast routing gives us the room for a selection out of some destinations. Denote the anycast group as $G(A)$, this paper deals with incomplete solution of the anycast selection for efficiency and scalability. We also denote the subset $\{x, y\} \subseteq G(A)$ as $\{x, y\}_A$. In order to appropriately characterize traffic both at ingress router and within the network, we use a general traffic descriptor in the form of traffic functions and their time independent constraint functions as described in [23].

The source traffic of flow is constrained by a leaky bucket with parameters pair (σ, ρ) . For simplicity, we define that all flows obey the uniform constraint such that $f(t) \leq \sigma + \rho t$. The total amount of traffic, generated by this source during any time interval $[t, t + I]$, is bounded by $\min\{CI, \sigma + \rho I\}$, i.e. for any $t > 0$ and $I > 0$, the total amount of an traffic $f_k(t)$ at source server k in time interval $[0, t)$ satisfies the following inequality:

$$f_k(t + I) - f_k(t) \leq \min\{CI, \sigma + \rho I\} \quad (1)$$

where σ is the burst size of source traffic for all flows, ρ is the average rate of source traffic of anycast flows.

The leaky bucket appropriately characterizes the source traffic at the ingress router. This characterization is not valid within the networks for the traffic is perturbed by queuing delays at servers as it traverses the network. In general, we characterize traffic at any servers in the network by traffic functions and their time independent counterpart. Traffic constraint functions is described as following:

Definition 1 The traffic function $f_{j,k}(t)$ means the amount of the traffic arriving at server k by the input link j during time interval $[0, t)$.

Traffic functions are cumbersome to handle and not of much help in admission control for they are time dependent. A time-independent traffic characterization is given by the traffic constraint function, which is defined as following:

Definition 2 The function $F_{j,k}(I)$ is called the constraint function of traffic function $f_{j,k}(t)$ if for any $t > 0$ and $I > 0$, the following inequality holds:

$$f_{j,k}(t + I) - f_{j,k}(t) \leq F_{j,k}(I) \quad (2)$$

We define the end-to-end deadline requirement of anycast traffic (s, G) to be D , where s is the source and G is the set of recipients of the anycast flow. Suppose an anycast flow $f_A(t)$ from source s is constrained by (σ_A, ρ_A) , i.e., $f_A(t + I) - f_A(t) \leq \min\{CI, \sigma_A + \rho_A I\}$ for all time t and duration I , thus denote $f_A(t) \sim (\sigma_A, \rho_A)$. All the flows through the network are guaranteed the same delay when they pass through the same server. In this paper, we use d_k to denote the local worst-case queuing delay suffered by traffic at server k , which has to pass through. In order to derive the computation formula of the local worst-case queuing

delay, we first assume that the traffic in the network system is empty before the network starts working.

4. Calculation of worst-case queuing delay

Based on the description of traffic function, we may derive the output traffic through the following lemma, which is important for the calculation of the delay when the traffic is intensive.

Lemma 1 *In time interval I , the aggregated traffic of the group of flows $G_{j,k}(I)$ where formed by all flows going through server k from input link j is constrained by*

$$F_{j,k}(I) = \begin{cases} CI & I \leq \tau_{j,k}(I) \\ n_{j,k}(I)[\sigma + \rho(I + Y_k)] + \sigma_A + \rho_A(I + Y_k) & I > \tau_{j,k}(I) \end{cases} \quad (3)$$

where $\tau_{j,k}(I) = \frac{n_{j,k}(I)[\sigma + \rho Y_k] + \sigma_A + \rho_A Y_k}{C - n_{j,k}(I)\rho - \rho_A}$, $n_{j,k}(I)$ is the maximal number of the other flows from source s arrived at sever k from input link j in time interval with length I . Y_k is the maximum of the worst-case delays experienced by all flows before arriving at server k , e.g. $Y_k = \max_{R \in S_k} \sum_{s \in R} d_s$, where S_k is the set of all paths passed by packets before arriving at server k .

Proof: For flow χ in the group of flows $G_{j,k}(I)$, let Y_k be the total worst-case queuing delay experienced by packets of an anycast flow χ before arriving at server k and H_x is the source traffic function of anycast flow χ , then $F_x(I) \leq H_x(I + Y_k) \leq \sigma + \rho(I + Y_k)$ or $F_x(I) \leq H_x(I + Y_k) \leq \sigma_A + \rho_A(I + Y_k)$ therefore $F_{j,k}(I) \leq \sum_{x \in G_{j,k}(I)} F_x(I) \leq n_{j,k}(I)[\sigma + \rho(I + Y_k)] + \sigma_A + \rho_A(I + Y_k)$, $F_{j,k}(I) \leq CI$ by using this inequality, we have

$$\begin{aligned} G_{j,k}(I) &\leq F_{j,k}(I) \leq \min\{CI, n_{j,k}(I)[\sigma + \rho(I + Y_k)] + \sigma_A + \rho_A(I + Y_k)\} \\ &= \begin{cases} CI & I \leq \tau_{j,k}(I) \\ n_{j,k}(I)[\sigma + \rho(I + Y_k)] + \sigma_A + \rho_A(I + Y_k) & I > \tau_{j,k}(I) \end{cases} \end{aligned} \quad \square$$

Lemma 2 *In time interval with length I , the aggregated traffic of the group of flows $G_k(I)$ formed by all flows input to server k is constrained by*

$$F_k(I) = \begin{cases} (L - i + 1)CI + \sum_{j=1}^{i-1} n_{j,k}(I)[\sigma + \rho(I + Y_k)] + i[\sigma_A + \rho_A(I + Y_k)], \\ \tau_{i-1,k}(I) \leq I \leq \tau_{i,k}(I), \quad i = 1, 2, \dots, L \\ \sum_{j=1}^L n_{j,k}(I)[\sigma + \rho(I + Y_k)] + [\sigma_A + \rho_A(I + Y_k)] & I > \tau_{L,k}(I) \end{cases} \quad (4)$$

where $\tau_{1,k}(I) \leq \tau_{2,k}(I) \leq \dots \leq \tau_{L,k}(I)$ and L is the maximal number of input edge to each server.

Proof: By using Lemma 1, we know that

$$G_k(I) \leq \sum_{j=1}^L F_{j,k}(I) = F_k(I)$$

$$= \begin{cases} (L-i+1)CI + \sum_{j=1}^{i-1} n_{j,k}(I)[\sigma + \rho(I + Y_k)] + i[\sigma_A + \rho_A(I + Y_k)] \\ \quad n_{i-1,k}(I) \leq I \leq n_{i,k}(I), \quad i = 1, 2, \dots, L \\ \sum_{j=1}^L n_{j,k}(I)[\sigma + \rho(I + Y_k)] + [\sigma_A + \rho_A(I + Y_k)] \quad I > n_{L,k}(I) \end{cases}$$

Thus, the lemma is proved. $\square$

Lemma 3 *There exists $I_0 > 0$ such that the local worst-case queuing delay d_k suffered by traffic at Server k satisfies the following inequality:*

$$d_k \leq \frac{[\sigma_A + \rho_A Y_k] + \sum_{j=1}^L n_{j,k}(I_0)[\sigma + \rho Y_k]}{C - \rho_A - \sum_{j=1}^L n_{j,k}(I_0)\rho} \quad (5)$$

Proof: It is easy to know the following inequality: $d_k \leq \frac{1}{C} \max_{I>0} \{F_k(I) - CI\}$, by Lemma 2, we have:

$$F_k(I) - CI = \begin{cases} (L-i)CI + \sum_{j=1}^{i-1} n_{j,k}(I)[\sigma + \rho(I + Y_k)] + i[\sigma_A + \rho_A(I + Y_k)] \\ \quad \tau_{i-1,k}(I) \leq I \leq \tau_{i,k}(I), \quad i = 1, 2, \dots, L \\ \sum_{j=1}^L n_{j,k}(I)[\sigma + \rho(I + Y_k)] - CI + [\sigma_A + \rho_A(I + Y_k)] \quad I > \tau_{L,k}(I) \end{cases}$$

Hence $\exists I_0 > 0$ such that $\max_{I>0} \{F_k(I) - CI\} = \frac{[\sigma_A + Y_k] + \sum_{j=1}^L n_{j,k}(I_0)[\sigma + \rho Y_k]}{C - \rho_A - \sum_{j=1}^L n_{j,k}(I_0)\rho} C$. Therefore, (5) is true. $\square$

Theorem 1 *Assume that the utilization of Server k does not exceed α_k . Thus, we have $\sum_{j=1}^L n_{j,k}(I)\rho + \rho_A \leq \alpha_k C$ and*

$$d_k \leq \frac{\alpha_k}{\rho(1 - \alpha_k)} [\sigma + \rho Y_k] + \frac{1}{(1 - \alpha_k)C\rho} [\rho\sigma_A - \sigma\rho_A] \quad (6)$$

Proof: Note that function $f(x) = \frac{xY_k + \sigma_A + \frac{\alpha_k C - \rho_A}{\rho} \sigma}{C - x}$ increases along with x in $[0, C]$. Applying Lemma 3 and the supposition, we know that:

$$d_k \leq \frac{\alpha_k C Y_k + \sigma_A + \frac{\alpha_k C - \rho_A}{\rho} \sigma}{C - \alpha_k C} = \frac{\alpha_k}{\rho(1 - \alpha_k)} [\sigma + \rho Y_k] + \frac{1}{(1 - \alpha_k) C \rho} [\rho \sigma_A - \sigma \rho_A]$$

□

It has been proved that when $\alpha_k \geq 0.62C$ for multistage and ring networks, then the delay may become unstable with the control of (σ, ρ) regulator [18].

Remark 1 From Theorem 1, it is easy to know that the network is stable if $\rho_A + \sum_{j=1}^L n_{j,k}(I) \rho \leq \alpha_k C$.

We note that the upper bound of Theorem 1 depends on Y_k . The value of Y_k , in turn, depends on the delays experienced at servers other than server k . In general, we have a circular dependency. Hence, the delay values depend on each other and must be computed simultaneously. We denote the upper bounds of delays by V -dimensional vector: $d = [d_1, d_2, \dots, d_V]^T$ and

$$\begin{aligned} \varphi(d) &= [\varphi_1(d), \varphi_2(d), \dots, \varphi_V(d)]^T, \quad \text{where} \\ \varphi_k(d) &= \frac{\alpha_k}{\rho(1 - \alpha_k)} [\sigma + \rho Y_k] + \frac{1}{(1 - \alpha_k) C \rho} [\rho \sigma_A - \sigma \rho_A], \quad k = 1, 2, \dots, V. \end{aligned} \quad (7)$$

Therefore the queuing delay bound vector can be determined by iterative methods for system of equations: $d = \varphi(d)$. Furthermore, once an anycast flow's path r is selected, the end-to-end delay d_{e2e} for each packet in this flow can be computed according following formula:

$$d_{e2e} = \sum_{k \in r} d_k \quad (8)$$

5. Efficient admission control algorithms for real-time anycast flows

Let triple $(s, G(A), R)$ denote the stream of flow requests from source s to anycast group $G(A)$ with routes set R formed by the shortest path from source s to one destination in $G(A)$. We assume that $G(A)$ contains K recipients. In this section, we will present a set of efficient admission control algorithms, taking special consideration to satisfy the real-time communications for AFs. Initially, the base-line anycast algorithm will be presented and then we extend the base-line algorithm to consider the incomplete routing information for an efficient and scalable admission control.

The elements in R are arranged according to their lengths in non decreasing order, e.g. $R = \{r_1, r_2, \dots, r_k\}$ with property $|r_1|, |r_2|, \dots, |r_k|$ where $|r_i|$ denotes the static distance (number of hops) of route r_i . For anycast flow (s, G, R) , we select destinations or route according to Biased Weight Assignment Algorithms (WD in short) as proposed in [11], e.g.

the weight W_i associated with destination i is

$$W_i = \frac{1/|r_i|}{\sum_{j=1}^K 1/|r_j|} \quad (9)$$

i.e. the probability of selecting i th destination is W_i for $i = 1, 2, 3, \dots, K$.

Let $(s_i, a_i, r_i)(i = 1, 2, \dots, M)$ denote the other flows (i.e., cross traffic) in the network from source s_i to destination a_i through route r_i and $F(A) = \{(s_i, a_i, r_i) | i = 1, 2, \dots, M\}$. Combining route selection algorithm and Theorem 1, we have the following basic admission control algorithm for a real-time anycast flow:

Algorithm 1: Real-Time Admission Control for Anycast Flow

Input: Number of nodes of the network and the set of all flows existing in the network. Let $F = \{(s_i, a_i, u_i) | i = 1, 2, \dots, M\}$ and the parameters (σ, ρ) constrain these flows from source s_i . An anycast flow request from source s to the set G of recipients by routes set $R = \{r_1, r_2, \dots, r_k\}$ with constraints parameters (σ_A, ρ_A) and deadline requirement to be D . Let the portion of link bandwidth assigned to traffics is α and iteration error tolerance ε . The number of input links of each node is L . The Capacity of each node is C .

Output: whether the anycast flow can be admitted into the network and give the destinations table for the anycast flow when it can be admitted.

Begin

1. $Z_1 := 1/(1-\alpha)$; $Z_2 := \frac{Z_1 \sigma_A}{C} + \frac{Z_1 \alpha \sigma}{\rho} + \frac{Z_1 \sigma \rho_A}{C \rho}$;

2. $T := 1$;

3. Select a destination A_j in G or a route r_j in R based on probability distribution W_j of the corresponding route r_j .

4. for $l=1, 2, 3, \dots, V$ do $d_l := Z_2$;

5. for $k=1, 2, 3, \dots, V$ do

$$5.1 \quad Y_k = \begin{cases} \sum_{l=1}^{m_k} d_{v_l} r_j = \langle s_h, v_1, v_2, \dots, v_{m_k}, k, \dots, A_j \rangle & \text{and} \\ r_j = \langle s, v_1, v_2, \dots, v_{m_i}, k, \dots, A_j \rangle; \\ 0 & k \notin r_j \end{cases}$$

5.2 for $h=1, 2, 3, \dots, M$ do

$$5.2.1 \quad Y_{k,h} = \begin{cases} \sum_{l=1}^{m_{k,h}} d_{v_l} r_h = \langle s_h, v_1, v_2, \dots, v_{m_{k,h}}, k, \dots, D_h \rangle; \\ 0 & k \notin r_h \end{cases}$$

5.2.2 if $Y_k < Y_{k,h}$ then $Y_k := Y_{k,h}$

5.3 $Z_k := \alpha Y_k + Z_2$;

6. if $\frac{\sum_{k=1}^V |Z_k - d_k|}{\sum_{k=1}^V d_k} \leq \varepsilon$ then goto 9

7. For $k=1, 2, \dots, V$ do $d_k := Z_k$;

8. goto 5;

```

9.  $E_k := \sum_{v_i \in r_j} d_i$ ;
10. If  $E_k > D$  and  $T < K$  then  $T := T + 1$  (search another path); go to 3; else report "Reject"; stop;
11. Report "Accept" and use path  $r_j$  and destination  $A_j$  for accepting anycast flow  $(s, G(A), R)$  with deadline requirement  $D$  satisfied;
end.

```

The essentiality of Step 3 in above algorithm is to transform the anycast flows to unicast flows by selecting a destination or path in corresponding addresses group. The selection of destination is based on weight-distance probability proposed in [24]. The destination selection of Step 4 to Step 8 is an iterative computation of the local delay at each nodes, taking all initial values (zero) under the selected route in Step 3. Step 9 tests if the anycast flow arrives at one of the corresponding recipient group in pre-limited time according the selected route in Step 3. When the end-to-end delay of anycast flow is beyond the corresponding pre-limited time, Step 10 continues to select a new route by using the multiple option of anycast communication until all possible routes combination is exhausted. When the results of the algorithm above give successful information, we find the route for the anycast flow request. Therefore, what we need to do is to test if the corresponding path r_j has enough bandwidth for the incoming anycast requirement at source s with the end-to-end deadline D . This admission control is scalable in the sense that the admission node makes distributed decision. However, in Algorithm 1 Step 10, the re-test of next path may cause further delay for the admission control of the anycast flow request. To make the algorithm feasible, we here propose a parallel version of Algorithm 1 by allowing multiple path setting up simultaneously:

Algorithm 2: Parallel Real-Time Admission Control

Input: (similar to Algorithm 1)

Output: whether the anycast flow can be admitted;

Begin

1'-9': Select K paths to invoke Algorithm 1 Steps 1 to 9 in parallel;

10': If any $E_j < D$ then Report "Accept" and use path r_j and destination A_j for accepting anycast flow $(s, G(A), R)$ with deadline requirement D satisfied. The other path admission control algorithm can be terminated;

11': If none of the path with $E_j < D$, then Report "Reject" because no path satisfied the deadline D .

End.

With the parallel version of admission control, our algorithm is much more efficient and the admission control algorithm can be applied to shorten the admission delay. The parallel admission control differs from multipath connections [5, 6] in the sense that different destinations are tried for the connection and the best one is chosen if it exists. This is unique feature of anycast flow where multiple servers can be targeted for the selection which can not be applied to unicast or multicast flows where the destinations have been pre-determined.

6. Experimental results

Network. The network considered in our experiments is ARPA network. Figure 2 shows the topology of the network. There are 56 nodes that are interconnected by links. Link bandwidth capacity is assumed to be 100 Mbps, and 40 percent of link bandwidth is reserved for anycast flows.

Traffic model. We first take the ARPA network as shown in Figure 1, as an example. The network consists of 21 nodes. Assume an anycast flow's source is $S = \{0\}$ and corresponding recipients (anycast) group $G = \{5, 9, 10, 13, 14, 15\}$, route set $R = \{\langle 0, 1, 5 \rangle, \langle 0, 1, 5, 9 \rangle, \langle 0, 2, 7, 10 \rangle, \langle 0, 1, 5, 9, 14 \rangle, \langle 0, 1, 5, 9, 14, 13 \rangle, \langle 0, 2, 7, 10, 15 \rangle\}$. There existing two flows from source 18 to 15 and 0 to 4, corresponding routes are $\langle 0, 1, 3, 4 \rangle$ and $\langle 18, 17, 15 \rangle$ respectively. Take $\sigma_A = \sigma = 640$, $\rho_A = \rho = 16$, $\alpha = 0.4$, $\varepsilon = 0.0000001$, $D = 100$, then $r1 = \langle 0, 1, 5 \rangle$ is a selection of routes and the anycast is admitted.

Our second performance is tested through 56 nodes of network as depicted in Figure 2. We assume that the sources of anycast flows is $S = \{1, 56\}$ and for each source, there are two anycast groups which consist of 8 members. One is $G_{1,1} = G_{56,1} = \{5, 9, 20, 23, 34, 35, 42, 53\}$, the others are $G_{1,2} = G_{56,2} = \{4, 8, 23, 25, 36, 39, 45, 55\}$. We also assume that the route from each source to a destination is fixed. We use $\langle p, \dots, q \rangle$ to denote a route from node p to node q . We defined the length of a route as the number of hops (i.e. nodes) on the route.

We have tested the following paths with NS-2 simulator.

$$\begin{aligned}
 R_{1,1} &= \{\langle 1, 2, 3, 4, 5 \rangle, \langle 1, 2, 3, 4, 5, 6, 9 \rangle, \langle 1, 2, 3, 4, 19, 20 \rangle, \langle 1, 2, 7, 14, 18, 24, 23 \rangle, \langle 1, \\
 &\quad 2, 7, 1, 18, 24, 29, 34 \rangle, \langle 1, 2, 3, 4, 19, 25, 30, 35 \rangle, \langle 1, 11, 37, 52, 53, 54, 55, 56, 49, 46, \\
 &\quad 42 \rangle, \langle 1, 11, 57, 52, 53 \rangle\} \\
 R_{1,2} &= \{\langle 1, 2, 3, 4 \rangle, \langle 1, 2, 3, 4, 19, 20, 15, 8 \rangle, \langle 1, 2, 7, 14, 18, 24, 23 \rangle, \langle 1, 2, 3, 4, 19, 25 \rangle, \\
 &\quad \langle 1, 2, 3, 4, 19, 20, 26, 47, 43, 36 \rangle, \langle 1, 11, 37, 52, 51, 40, 39 \rangle, \langle 1, 2, 3, 4, 19, 20, 26, 31, \\
 &\quad 42, 41, 45 \rangle, \langle 1, 11, 37, 52, 53, 54, 55 \rangle\} \\
 R_{56,1} &= \{\langle 56, 50, 47, 26, 20, 19, 4, 5 \rangle, \langle 56, 50, 47, 26, 20, 15, 8, 9 \rangle, \langle 56, 50, 47, 26, 20 \rangle, \\
 &\quad \langle 56, 55, 54, 53, 52, 51, 40, 39, 38, 34, 29, 24, 23 \rangle, \langle 56, 55, 54, 53, 52, 51, 40, 39, 38, \\
 &\quad 34 \rangle, \langle 56, 55, 54, 53, 52, 51, 40, 35 \rangle, \langle 56, 49, 46, 42 \rangle, \langle 56, 55, 54, 53 \rangle\}
 \end{aligned}$$

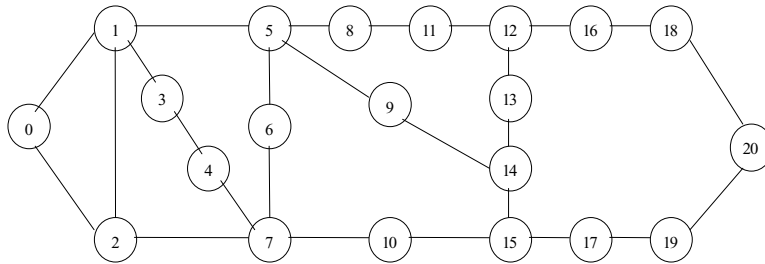


Figure 1. Simple ARPA network.

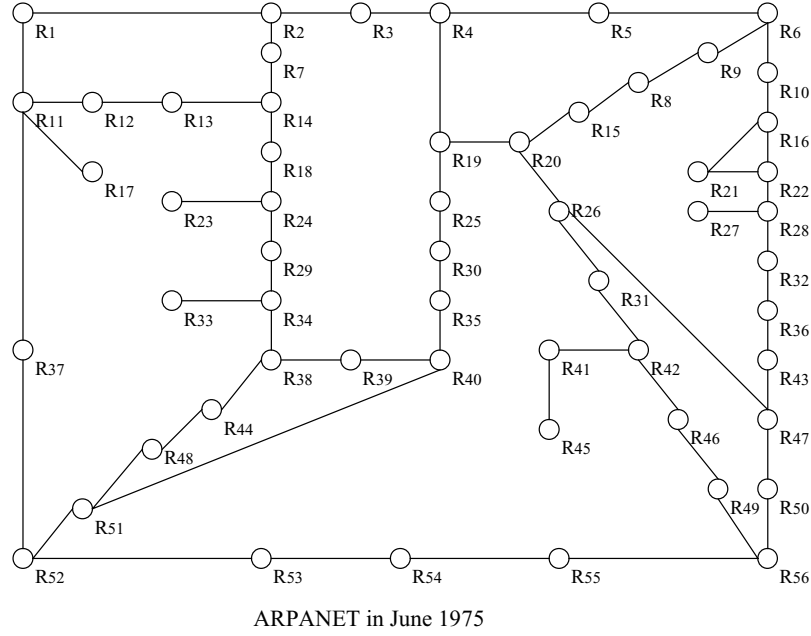


Figure 2.

$R_{56,2} = \{ \langle 56, 50, 47, 26, 20, 19, 4 \rangle, \langle 56, 50, 47, 26, 20, 15, 8 \rangle, \langle 56, 55, 54, 53, 52, 51, 40, 39, 38, 34, 29, 24, 23 \rangle, \langle 56, 50, 47, 26, 20, 19, 25 \rangle, \langle 56, 50, 47, 43, 36 \rangle, \langle 56, 55, 54, 53, 52, 51, 40, 3 \rangle, \langle 56, 49, 46, 42, 41, 45 \rangle, \langle 56, 55 \rangle \}$ with the delay as shown in the following tables.

Based on above simulation and data, we may observe that our parallel admission control algorithms through delay bound calculation can be very efficient. As the shortest delay bound path can be chosen for the admission of a new flow. Selecting the flow destination has been the unique feature for anycast flow admission connections. From Tables 1 and 2,

Table 1. $\sigma = 640$ and the delay(s) varied with ρ for source node 1. It can be seen that the path $\langle 1-53 \rangle$ incurs the minimum delay

Path delay and rate ρ	1-5	1-9	1-20	1-23	1-34	1-35	1-42	1-53
32	66.5	99.5	83.2	99.8	116.5	114.9	166.3	[66.1]
64	33.4	49.8	42.3	49.8	58.3	57.5	83.2	[33.1]
128	16.8	24.5	21.2	25.1	29.5	28.6	42.3	[16.7]
256	8.5	12.3	11.0	12.5	14.6	14.3	21.2	[8.4]
512	4.5	6.4	5.6	6.4	7.5	7.2	11.2	[4.3]
1024	2.4	3.3	2.9	3.3	3.9	3.5	5.7	[2.3]

Table 2. $\sigma = 640$ and the delay(s) varied with ρ for source node 56. It can be seen that the path (56–55) incurs the minimum delay

Path/delay/rate ρ	56–4	56–8	56–23	56–25	56–36	56–39	56–45	56–55
32	99.8	101.2	199.8	99.6	67.4	116.3	83.4	[16.6]
64	49.5	50.2	101.1	48.6	33.6	58.2	41.6	[8.3]
128	24.9	25.1	50.1	24.5	16.7	29.1	21.1	[4.2]
256	12.5	12.6	25.1	12.2	8.3	14.9	11.1	[2.1]
512	6.2	6.3	12.6	6.1	4.2	7.5	5.1	[1.1]
1024	3.1	3.2	6.3	3.1	2.1	3.7	2.6	[0.6]

it can be seen that the path with the shortest delay bound is chosen which are (1–53) and (56–55) respectively.

7. Conclusion

We have studied the calculation for end-to-end delay bounds of real-time anycast flows and presented a parallel real-time admission control algorithm for effective anycast flow admission and connections. This will effectively shorten the admission delay and enable us to find a good quality of path with the shortest delay bound. This work is particularly important for flow control where the replicated servers can be selected. Our approach focuses on the deterministic guarantees of delay bound analysis based on network calculus [7, 8]. However, our approach extends the network calculus with adaptive approach that multiple paths to the different destinations or servers may be evaluated. Thus, our algorithm may gain the efficient connection for finding a short delay path that satisfies the real-time constraints of anycast flow requests. We have achieved the experiment data in the two APAR networks and shown that the end-to-end delay can be effectively calculated.

References

1. G. Apostolopoulos, D. Williams, S. Kamat, R. Guerin, A. Orda, and T. Przygienda. QoS Routing Mechanisms and OSPF Extensions. *RFC 2676, IETF*, 1999.
2. ATM Forum. Private network-network interface specification v.1.0. Tech. rep. af-pnni-0055.000, ATM Forum, March 1996.
3. S. Bhattacharjee, M. H. Ammar, E. W. Zegura, V. Shah, and Z. Fei. Application-layer anycasting. *Proceedings of IEEE INFOCOM'97*, April 1997.
4. B. Choi, D. Xuan, C. Li, R. Bettati, and W. Zhao. Scalable QoS guaranteed communication services for real-time. *Proceeding of IEEE ICDCS*, 2000.
5. I. Cidon, R. Rom, and Y. Shavitt. Multi-path routing combined with resource reservation. *Proceedings IEEE INFOCOM '97*, 7–12, April 1997, vol. 1, pp. 92–100.
6. I. Cidon, R. Rom, and Y. Shavitt. Analysis of multi-path routing. *IEEE/ACM Transactions on Networking*, 7(6):885–896, 1999.
7. R. L. Cruz. A calculus for network delay, Part I. *IEEE Transactions on Information Theory*, vol. 37, Jan. 1991.
8. R. L. Cruz. A calculus for network delay, Part II. *IEEE Transactions on Information Theory*, vol. 37, Jan. 1991.

9. A. Dailianas and A. Bovopoulos. Real-time admission control algorithms with delay and loss guarantees in ATM networks. *IEEE INFOCOM*, 1994.
10. S. Deering and R. Hinden. Internet Protocol version 6 (Ipv6) specifications, RFC2460, Dec. 1998.
11. S. Deering and R. Hinden. Internet Protocol Version 6 (IPv6) Specification. RFC 2460, Dec. 1998.
12. V. Firoiu, J. Kurose, and D. Towsley. Efficient admission control for EDF schedulers. *IEEE INFOCOM*, 1998.
13. R. Guerin, A. Orda, and D. Williams. QoS routing mechanisms and OSPF extensions. *Proc of IEEE GLOBE-COM97*, Phoenix, Arizona, 1997, vol. 3, pp. 1903–1908.
14. J. D. Guyton and M. F. Schwartz. Locating nearby copies of replicated Internet servers. *Proc. SIGCOMM'95*, Oct. 1995, pp. 288–298.
15. F. Hao, E. W. Zegura, and M. H. Ammar. QoS routing for anycast communications: Motivation and an architecture for DiffServ networks. *IEEE Communications Magazine*, June 2002, pp. 48–56.
16. R. Hinde and S. Derring. IP Version 6 Addressing Architecture, RFC 1884, Dec. 1995.
17. R. Hinde and S. Deering. IP Version 6 Addressing Architecture. RFC 1884, Dec. 1995.
18. W. Jia, H. Wang, M. Tang, and W. Zhao. Effective delay control for high rate heterogeneous real-time flows. *Proceedings IEEE ICDCS*, May 19–22, 2003, Providence, Rhode Island, USA.
19. W. Jia, D. Xuan, and W. Zhao. Integrated routing algorithms for anycast messages. *IEEE Communications Magazine*, 38(1):48–53, 2000.
20. W. Jia, G. Xu, and W. Zhao. Integrated fault-tolerant multicast and anycast routing algorithms. *IEEE Proceeding of Computer Digital Technology*, 147(4):266–274, 2000.
21. D. Katabi and J. Wroclawski. A framework for scalable global IP-anycast (GIA). *Proc. of SIGCOMM'00*, Aug. 2000.
22. S. Wang, D. Xuan, R. Bettati, and W. Zhao. Proving absolute differentiated services for real-time applications in static-priority scheduling networks. *IEEE INFOCOM*, 2001.
23. D. Xuan and W. Jia. Distributed admission control for anycast flows with QoS requirements. *Proceedings IEEE ICDCS*, Phoenix, USA, April 16–19, 2001, pp. 292–300.
24. D. Xuan, W. Jia, W. Zhao, and H. Zhu. A routing protocol for anycast messages. *IEEE Transactions on Parallel and Distributed Systems*, 11(6):571–588, 2000.
25. E. W. Zegura, M. H. Ammar, Z. Fei, and S. Bhattacharjee. Application-layer anycasting. *IEEE/ACM Transactions on Networking*, 8(4):455–466, 2000.